Informe de Laboratorio #2

Nombres:

- Lucas Ansieta M. - 202373504-3
- Bastián Garcés C. - 202373510-8
- Benjamín Palacios T. - 202373532-9

Preguntas de desarrollo

1. ¿Qué es DNS y cuál es su origen? ¿Qué pasaría si los servidores DNS no existieran o no se hubieran implementado?

DNS, por sus siglas “Domain Name System”, es un sistema descentralizado de nomenclatura utilizado por dispositivos conectados a cualquier red IP, como por ejemplo, internet. Su principal función es la resolución de nombres inteligibles para las personas (llamados Nombres de Dominio o Domain Names) a direcciones IP. Dicho de otra forma, DNS actúa como un “traductor” que las máquinas usan para resolver Nombres de Dominio y encontrar Direcciones IP.
DNS nace de la necesidad de las personas para recordar de una manera sencilla los nombres de todos los servidores conectados a internet. Antes de la implementación de los DNS, el SO de cada dispositivo almacenaba localmente un archivo de texto llamado “HOSTS”, que contenía el registro de todos los nombres de dominio conocidos. El problema es que cuando la red empezó a crecer de manera exponencial, se descubrió que un sistema centralizado para resolver nombres de dominio era poco práctico e imposible de mantener al largo plazo, por esta razón, en 1984 es emitido oficialmente el RFC 920, documento que define políticas y requisitos para establecer dominios organizacionales, sustituyendo el sistema centralizado por lo que hoy conocemos como DNS (NOTA: El archivo “HOSTS” aún existe en los SO’s, pero ya no se usa como método principal para resolver nombres de dominio, pues para eso están los DNS).
Si actualmente no existieran los servidores DNS, las personas no solo deberían memorizar las IP de todas las páginas que quieran visitar… Sino que además, si a una empresa se le ocurre cambiar sus servidores o su ISP, junto a eso, también cambiarían sus direcciones IP, por lo que existiría el problema de que lo ya memorizado no perdura en el tiempo y habría que estar memorizando IPs constantemente.
Lógicamente, para nosotros es muchísimo más fácil memorizar “google.com” que memorizar “173.194.215.139”, y aunque lográramos memorizar dicha IP, nada nos asegura que de acá a 2 semanas esa IP para acceder a Google sea la misma, por lo que la existencia de los servidores DNS ayuda a que el establecimiento de la comunicación entre nuestros dispositivos con los servidores donde se alojan todos los sitios web que visitamos a diario sea mucho más amena y amigable para nosotros.

2. Explique la diferencia entre el servidor que acaba de construir y un servidor del tipo “recursivo”. ¿Cómo podría modificar su servidor para que sea de este tipo?

El servidor desarrollado en la experiencia práctica actúa como un servidor básico con respuestas estáticas. Su funcionamiento se limita a escuchar peticiones en el puerto DNS (UDP 53) y responder con una dirección IP estática predefinida cada vez que se le consulta por el **dominio objetivo**. Lo hace estructurando manualmente el paquete DNS de respuesta mediante la función build_response(data), donde fija *flags* de respuesta y adjunta los bytes de la IP. Si se le consulta por un dominio distinto al que tiene configurado, este servidor no sabrá cómo responder, ya que carece de mecanismos para buscar información externa.

Por el contrario, un **servidor DNS recursivo** asume la responsabilidad completa de resolver el dominio en nombre del cliente. Si un servidor recursivo recibe una consulta sobre un dominio que no tiene en su caché local, actuará él mismo como cliente y comenzará a realizar peticiones iterativas a través de la jerarquía global de DNS (consultando primero a los servidores Raíz, luego a los servidores TLD correspondientes, y finalmente a los servidores Autoritativos del dominio) hasta encontrar la IP exacta, la cual finalmente se la devuelve al cliente original.

Cómo modificar el servidor actual para que sea de tipo recursivo

Para transformar el script DNS_server.py en un servidor recursivo, se deberían implementar los siguientes cambios lógicos y estructurales en el código:

1. **Análisis dinámico de la consulta (Parsing):** En lugar de inyectar directamente la IP_RESPUESTA, el servidor debe extraer y decodificar el nombre de dominio solicitado desde la sección de preguntas del paquete original que entra al socket.
2. **Implementación de un cliente DNS interno:** Se debe instanciar un nuevo socket UDP dentro de la función de manejo del servidor. Este socket tomará el paquete de consulta original y lo reenviará hacia los servidores de la jerarquía DNS.
3. **Resolución iterativa:** El código debe incluir la lógica para recibir las respuestas de estos servidores externos. Si la respuesta es una derivación hacia otro servidor, el script debe iterar y consultar a la nueva IP proporcionada hasta obtener el registro "A" definitivo.
4. **Reenvío y Caché:** Una vez obtenida la IP real, el servidor debe tomar esa respuesta, asegurar que el identificador de transacción coincida con el de la petición del cliente original, y enviársela de vuelta. Para que sea un servidor recursivo eficiente, también se debería programar una estructura de datos que almacene temporalmente estas resoluciones para responder inmediatamente a futuras consultas idénticas.

3. ¿Qué tipo de registro y consulta se utilizaron para este caso? ¿Por qué no se utilizaron otros tipos? Fundamente en base a los contenidos vistos y genere una tabla con los tipos de registros DNS.

En el laboratorio se utilizó una **consulta estándar** para solicitar un **registro de tipo A**.

Esto queda en evidencia en el código del servidor (DNS_server.py), donde la respuesta se construye para entregar los bytes correspondientes a una dirección IPv4 frente a la consulta por un dominio objetivo. Según los contenidos vistos en clases, el registro tipo A es precisamente el encargado de mapear un *hostname* a una dirección IP.

¿Por qué no se utilizaron otros tipos?

El objetivo final de la experiencia práctica es que el cliente se conecte exitosamente a un servidor central a través de un socket TCP. Para que esta conexión se establezca, el cliente necesita conocer obligatoriamente la dirección IP del destino.

No se utilizaron otros tipos de registros (como los vistos en clases) por las siguientes razones:

- Si se hubiese utilizado un registro **CNAME**, el servidor habría devuelto un nombre canónico (alias) en lugar de la IP. El cliente habría tenido que hacer una segunda consulta DNS para resolver ese nuevo nombre, lo cual excede el alcance del cliente básico construido.
- Si se hubiesen utilizado registros **NS** o **MX**, el servidor habría devuelto el nombre de un servidor de nombres autorizados o el nombre de un servidor de correo, respectivamente. Ninguno de estos valores le sirve al cliente para establecer su conexión TCP con el servidor del laboratorio.

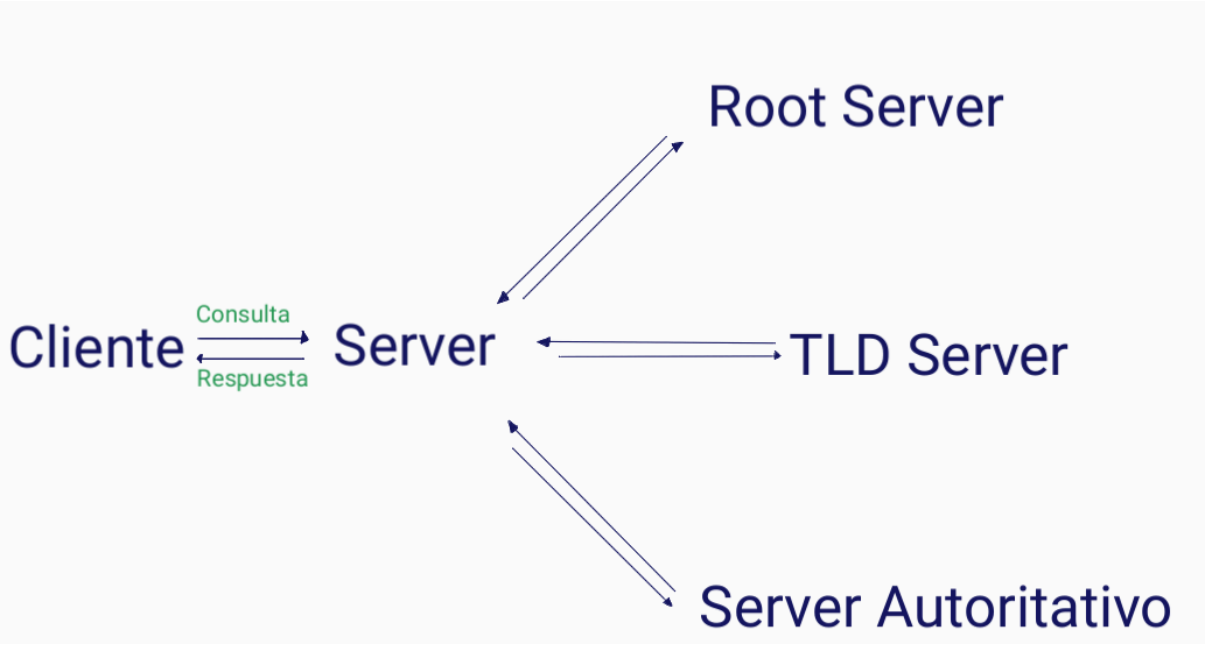
Tabla de tipos de registros DNS

En base a la estructura de los Resource Records enseñada en el Capítulo 2 de la asignatura, los registros se componen del formato (Name, Value, Type, TTL). A continuación, se presenta la tabla con los tipos vistos:

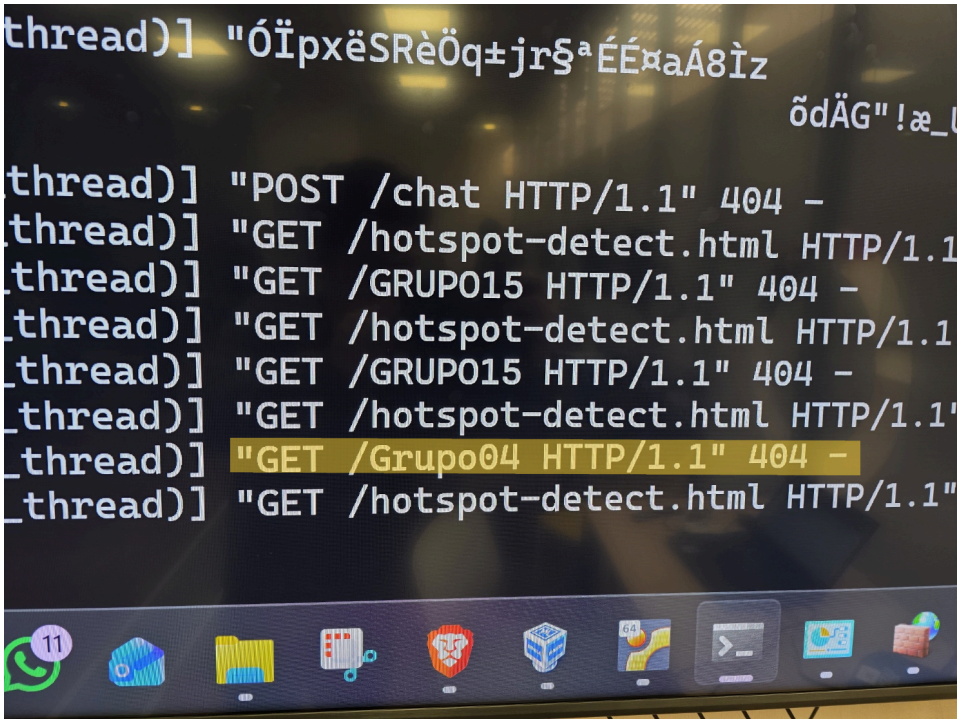
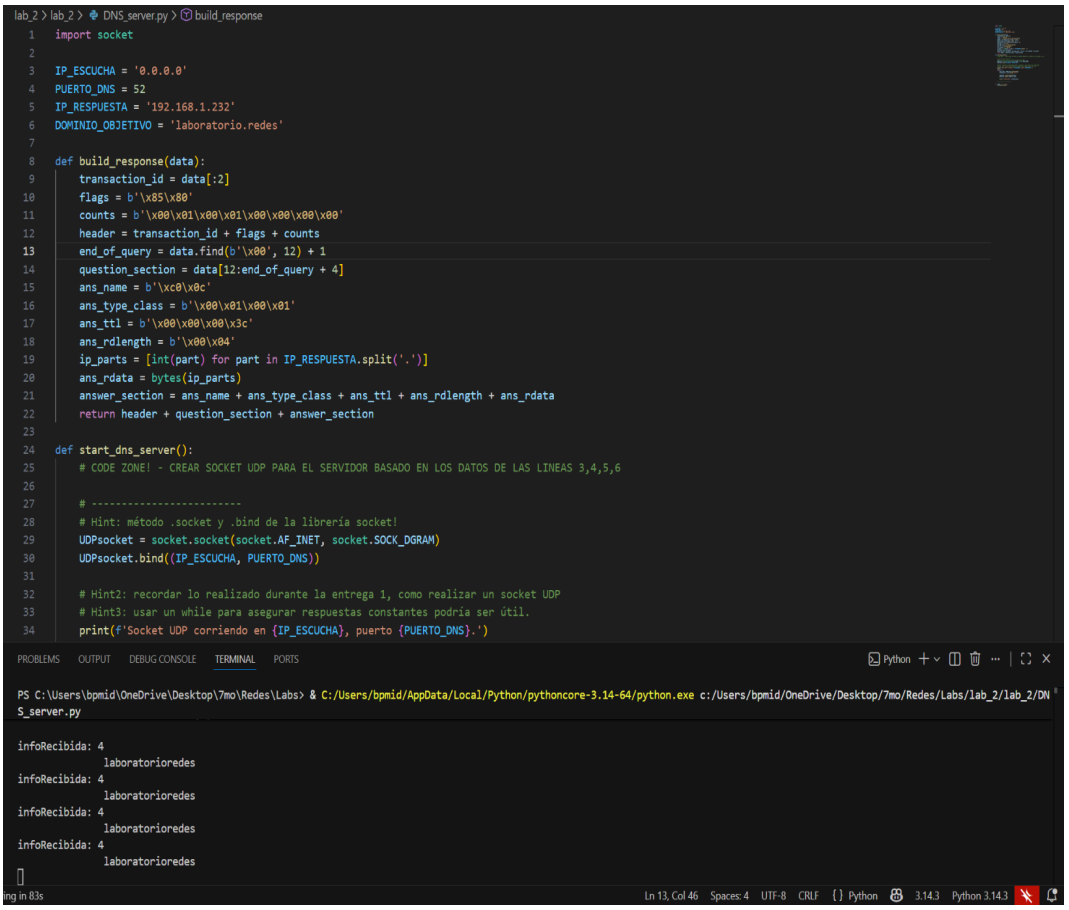
Tipo	Significado del campo Name	Significado del campo Value	Ejemplo de uso
A	Es el *hostname* (nombre del host).	Es la dirección IP.	(relay1.bar.foo.com, 145.37.126, A)

NS	Es el dominio (ej. foo.com).	Es el *hostname* de autorización del servidor de nombres para ese dominio.	(foo.com, dns.foo.com, NS)
CNAME	Es el nombre alias.	Es el nombre canónico (el real).	(www.ibm.com, servereast.backup2.ibm.com, CNAME)
MX	Es el dominio (asociado implícitamente).	Es el nombre del servidor de correo electrónico asociado con el *name*.	(foo.com, mail.bar.foo.com, MX)

4. Genere un diagrama de la interacción del laboratorio y compárela con la realidad, es decir, cómo se resuelve una consulta HTTP en un navegador a través de los servidores DNS.



A diferencia del laboratorio donde la comunicación fue directa, en la realidad el cliente hace una consulta al servidor DNS de su proveedor (recursivo). Luego, este servidor DNS consulta de forma iterativa a los Root Servers, a los servidores TLD y

al Servidor Autoritativo, y cada uno le responde indicando el siguiente paso hasta encontrar la IP exacta. Finalmente, el servidor DNS le responde al cliente con la IP final para que el navegador pueda establecer la conexión HTTP.

Evidencia de presencialidad en la experiencia